

DSL LenguajeDatos xd

Manual Técnico de Librerías, Sintaxis y Ejecución

Programa de Ciencias de la Computación e Inteligencia Artificial

Universidad Sergio Arboleda - Semestre 2026-2

1. Cómo Funcionan las 3 Librerías Propias

El proyecto cuenta con tres librerías desarrolladas desde cero en Python puro, sin dependencias de terceros (como NumPy, Pandas o Matplotlib). Cada librería responde al sufijo `_xd` en sus funciones.

A. `matematica_propia` (Reemplazo de NumPy)

Maneja vectores numéricos mediante la clase `Arreglo`. Permite realizar aritmética elemento a elemento, máscaras booleanas para filtrado y reducciones estadísticas.

```
arr = Arreglo([10, 20, 30, 40])
prom = arr.promedio_xd()          # 25.0
desv = arr.desviacion_estandar_xd() # 12.91
mascara = arr.mayor_que_xd(20)    # [False, False, True, True]
```

B. `datos_propios` (Reemplazo de Pandas)

Ofrece almacenamiento y manipulación de datos tabulares mediante las clases `Serie` y `TablaDatos`. Permite carga/escritura CSV, filtrado, selección de columnas, ordenamiento y agrupamiento.

```
tabla = cargar_csv_xd("datos.csv")
tabla["total"] = tabla["unidades"].arreglo * tabla["precio"].arreglo
sub = tabla.seleccionar_xd(["ciudad", "total"])
res = tabla.agrupar_por_xd("ciudad").resumir_xd(total_ventas=("total", "suma"))
guardar_csv_xd(res, "salida.csv")
```

C. `graficos_propios` (Reemplazo de Matplotlib)

Genera gráficos en formato SVG vectorial utilizando la clase `Figura`. Soporta 5 tipos de gráficos principales.

```
fig = crear_figura_xd(titulo="Ventas xd", etiqueta_x="Ciudad", etiqueta_y="Ventas")
fig.graficar_barras_xd(["Bogota", "Medellin"], [3500, 2800])
fig.guardar_svg_xd("salidas/barras.svg")
```

2. Sintaxis del DSL (LenguajeDatos)

El DSL es declarativo y utiliza el operador tubería (`|>`) para encadenar transformaciones de datos. Todas las funciones principales finalizan con el sufijo `_xd`.

Estructura de un Programa en el DSL:

```
# 1. Cargar archivo CSV
ventas = cargar_csv_xd "datos/ventas_prueba.csv"
```

```
# 2. Encadenar transformaciones con |>
ventas_limpias = ventas
|> seleccionar_xd [fecha, ciudad, categoria, unidades, precio]
|> filtrar_xd unidades > 10
|> crear_columna_xd total = unidades * precio

# 3. Agrupar y resumir
resumen = ventas_limpias
|> agrupar_por_xd [ciudad]
|> resumir_xd total_ventas = suma(total), promedio_u = promedio(unidades)

# 4. Guardar resultado y visualizar
guardar_csv_xd resumen "salidas/resumen_ciudades_dsl.csv"
graficar_barras_xd resumen titulo "Ventas por Ciudad xd" guardar "salidas/grafico.svg"
```

3. Cómo Ejecutarlo ("Runearlo")

Paso 1: Ejecutar un programa .dsl desde la Consola CLI

Abre la terminal en la raíz del proyecto y ejecuta el archivo CLI del intérprete:

```
python ejecutar_dsl.py ejemplos/programa_correcto1.dsl
```

Paso 2: Ejecutar las Pruebas Unitarias Automatizadas

Para validar la correctitud del motor interno y del análisis sintáctico ANTLR4:

```
python -m unittest tests/test_corte1.py
```

Paso 3: Probar el Diagnóstico de Errores Sintácticos

Al ejecutar un archivo con errores sintácticos, el intérprete reporta la línea y columna exactas:

```
python ejecutar_dsl.py ejemplos/programa_incorrecto.dsl
```